Auto Performance Profiler Pro - Detailed Documentation

Welcome to the comprehensive technical documentation for **Auto Performance Profiler Pro**. This document covers advanced usage, underlying architecture, API details, and instructions on how to extend the tool for your studio's specific needs.

📑 Table of Contents

1. Installation & Quick Start
2. Architecture Overview
3. Data Collection Module & Hardware Budgets
4. Visual Diagnostics & Snapshots
5. Offline Scene Analyzer (Editor)
6. The Auto-Fix Pipeline
7. Analytics Parsing & AI Suggestions
8. Enterprise Features (CI/CD Guardian, Wireless, & Comparator)
9. Advanced Editor Tooling (Heatmaps, Export, Treemaps, & Ghost Scanner)
10. API Reference & Extending the Tool
11. Performance Overhead Constraints
12. Troubleshooting & F.A.Q.

1. Installation & Quick Start

Installation

1. Import the `AutoPerformanceProfilerPro.unitypackage` into your Unity project.
2. The package will automatically compile under the `AutoPerformanceProfiler` namespace.
3. *Note: Ensure your project uses Unity 2021.3 LTS or newer to fully support the `Unity.Profiling` unmanaged API.*

The 3-Step Workflow ⏱️

To get immediate value out of the tool, follow this simple workflow:

1. **Analyze Offline:** Open the Tool via `Window -> Analysis -> Auto Performance Profiler Pro`. Click the **Run Offline Scene Scan** button. This instantly finds 30+ architectural mistakes before you even press play.
2. **Auto-Fix Bottlenecks:** Review the generated list of offenders. For any item with a magic wand icon, click ✨ **Auto-Fix** to let the tool instantly restructure the component natively.
3. **Capture Runtime Data:** Enter Play Mode. Click ⏺ **Record**. Play your game normally for 60 seconds (shoot enemies, open UIs). Click ⏹ **Stop**. Read the AI-generated paragraph at the top of the report to see exactly what scripts or graphics are lagging your game.

---

2. Architecture Overview

Auto Performance Profiler Pro is strictly divided into two primary assemblies to ensure zero overhead in your production builds:

- **Runtime Assembly (`AutoPerformanceProfiler.Runtime`)**: Contains the `PerformanceTracker` MonoBehaviour and the serializable `ProfilerReport` ScriptableObject data structure.
- **Editor Assembly (`AutoPerformanceProfiler.Editor`)**: Contains the `ProfilerWindow` UI, drawing logic, export functionality (CSV/JSON), and the `ProfilerAnalyzerExtensions` offline scanner.

This separation ensures that any Editor-only API calls (like `AssetDatabase` or `EditorGUILayout`) do not break your mobile or console builds.

---

3. Data Collection Module & Hardware Budgets

The core data collection is handled by `PerformanceTracker.cs`.

Hardware Budget Profiles

Instead of manually typing in threshold requirements per session, the tool uses `HardwareBudgetProfile` scriptable objects (right-click -> Create -> Auto Profiler -> Hardware Budget Profile). You can define exact max frame times (ms), max VRAM caps, and strict GC thresholds for different platforms (e.g. `Mobile Low-End` vs `PC Ultra`). Pass this asset into the PerformanceTracker to dynamically shift how warnings and Auto-Fix rules apply.

Setting Budgets Programmatically: If your game has a dynamic settings menu, you can inject the exact hardware profile at runtime before calling the profiler:

```
// Example of assigning a specific Mobile budget when the game initializes
[SerializeField] private HardwareBudgetProfile mobileLowEndProfile;
[SerializeField] private PerformanceTracker profiler;

void Start() {
    if (SystemInfo.systemMemorySize < 4000) {
        profiler.activeHardwareBudget = mobileLowEndProfile;
        Debug.Log("Switched Profiler Hardware Limits to Mobile Specification.");
    }
}
```

The `ProfilerRecorder` API

Unlike older Unity profiler methods that caused massive Garbage Collection (GC) spikes just by reading the data, this tool uses Unity's modern `Unity.Profiling.ProfilerRecorder` C++ struct API. It safely pulls metric data from Unity's internal unmanaged memory, meaning the act of profiling your game costs virtually **0 CPUms** and **0 Bytes of GC**.

Monitored Subsystems:

- **Main Thread C#**: Raw update loop logic time (`ProfilerCategory.Internal`).

- **Scripts (BehaviourUpdate)**: Explicit time spent executing user-created `MonoBehaviour.Update()` methods.
- **Render Thread CPU**: Time spent preparing graphic data for the GPU.
- **Physics processing**: Time spent computing Rigidbodies, Colliders, and Raycasts.
- **Director.Update (Animation)**: CPU time spent skinning meshes and calculating Animator state machines.
- **System Memory & VRAM**: Tracks total allocated RAM alongside Texture/Mesh specific Video RAM (crucial for mobile thermal throttling).

Periodic Object Leak Detection

Every 10 seconds (default), the `PerformanceTracker` executes an internal polling of the `GameObject` and `MonoBehaviour` counts in the active scene. It saves these as `ObjectSnapshot` data points. If the array size grows continuously over a 5-minute session, the tool classifies it as a Critical Memory Leak.

---

4. Visual Diagnostics & Snapshots

Identifying that a lag spike occurred at second 45 is useless if you don't know *what* the player was looking at.

Spike Thresholds

When `takeScreenshotsOnSpike` is enabled, the tracker monitors frame times against `fpsThreshold` and memory allocations against `gcThresholdBytes`.

If a threshold is breached, it utilizes `ScreenCapture.CaptureScreenshot` to take an immediate render of the Game View. These are localized in your machine's `Application.persistentDataPath/ProfilerScreenshots`.

Performance Footprint of Snapshots: *Writing a PNG to disk is inherently slow.* To prevent the Profiler *itself* from causing lag spikes while capturing screenshots, the tool downscales the resolution of the snapshot depending on your hardware before running the main-thread encode. The IO path is fully asynchronous where possible. When viewing the data in the Editor window, the custom GUI loads these `.png` files via direct `byte[]` reading to instantly display them in an interactive Texture2D box without importing them through the heavy Unity Asset Pipeline.

---

5. Offline Scene Analyzer (Editor)

A unique feature of this tool is the ability to locate severe performance bottlenecks *without ever pressing Play*. This is powered by `ProfilerAnalyzerExtensions.cs`.

How it Scans:

When you click **"Run Offline Scene Scan"**, the tool uses `Object.FindObjectsByType<T>(FindObjectsSortMode.None)` to iterate through the entire scene hierarchy. `FindObjectsSortMode.None` is utilized because it skips the internal InstanceID sorting, resulting in 3x faster scan times on massive AAA scenes compared to traditional `FindObjectsOfType`.

Severity Classifications

The scanner assigns one of three classifications to every bottleneck it finds:

- 🔴 **High Severity:** Critical issues that will unconditionally cause app crashes, massive framerate drops (e.g. Realtime Point Shadows), or out-of-memory errors on target devices. The CI/CD Guardian will block builds matching this severity.
- 🟠 **Medium Severity:** Poor architectural choices that create cumulative "death by a thousand cuts" CPU drag (e.g. leaving Rigidbody Interpolation active on a crate).
- ⚪ **Low Severity:** Micro-optimizations or memory warnings that may only affect strict mobile targets (e.g. Read/Write enabled meshes).

Implicit Rules Applied (30+ Targets):

1. **Empty C# Update Loops:** Reflection scanner finding custom MonoBehaviours that declare `Update()` but are empty, causing native-to-managed CPU overhead.
2. **Missing Object Pools:** Tracking arrays of `(Clone)` GameObjects indicating `Instantiate()/Destroy()` GC abuse instead of proper pooling.
3. **Realtime Point Light Shadows:** Finding Point or Spot lights calculating 6-pass depth shadows every frame.
4. **Particle System Overdraw:** Catching particle systems with Max Particles exceeding 5000 causing GPU quad overdraw.
5. **Duplicate EventSystems:** Flagging multiple EventSystems in a single scene destroying UI Input polling.
6. **Massive BoxCollider Broadphase:** Identifying astronomically large BoxColliders that force the physics engine to overlap-test the entire level.
7. **Dynamic Non-Convex Colliders:** Flagging MeshColliders that are not marked convex but have dynamic Rigidbodies attached (Crash risk).
8. **Rigidbody Interpolation Wastes:** Finding background physics props with CPU interpolation enabled (Should only be Player/Camera).
9. **Legacy UI Text:** Identifying usage of blurry, GC-heavy `UnityEngine.UI.Text` instead of modern TextMeshPro.
10. **UI MipMap Memory Waste:** Detecting 2D UI Sprites bypassing ScreenSpace with MipMaps enabled (Wastes 33% VRAM).
11. **Audio Load Freezes:** Finding >20s AudioClips missing "Load in Background" causing SSD thread halting.
12. **Multiple AudioListeners:** Finding scenes with >1 AudioListener creating engine warning loops.
13. **Audio 3D Stereo Wastes:** 3D Spatialized Audio imported in Stereo (costing 2x memory natively).
14. **Audio Compression Spikes:** Long SFX incorrectly set to "Decompress on Load".
15. **Pixel Perfect UI:** Canvas elements halting the main thread by attempting to snap to screen-pixels every frame.
16. **Missing Static Batching:** `MeshRenderer` without internal movement but lacking the `isStatic` flag.
17. **Missing LODs:** High-poly meshes lacking an `LODGroup` wrapper.
18. **Texture VRAM Bloat:** Material textures encoded at 4096px or larger.
19. **Reflection Probe Chokes:** Realtime probes set to refresh `EveryFrame` (re-rendering the scene 6x).
20. **Camera Far-Clip Limits:** Cameras rendering outwards past Z-5000 units causing Z-Buffer errors.
21. **UI RaycastTarget Bloat:** UI elements with Raycast active but no actual selectable script attached.
22. **Multi-Material Draw Calls:** Utilizing more than 2 distinct materials on a single mesh causing raw draw call additions.
23. **Canvas Rebuild Triggers:** Finding ScreenSpace overlays utilizing >10 layout groups.

24. **Mesh Read/Write Wastes:** `isReadable` enabled on static meshes, doubling memory footprints.
25. **God Objects:** Single `GameObjects` carrying more than 10 C# components.
26. **Project Architecture Backend:** Ensuring IL2CPP is activated instead of Mono via the modern `NamedBuildTarget` API.
27. **Mobile Software VSync:** Locating artificial input drag on mobile platforms. *...And more dynamically added structural integrity checks.*

---

6. The Auto-Fix Pipeline

When a rule is triggered by the Offline Scanner, it generates an `ObjectOffender` struct mapping the specific GameObject name, the risk severity, and the recommended solution.

Automated Refactoring

The **"✨ Auto-Fix"** button routes to `ProfilerAnalyzerExtensions.AutoFixSpecificOffender()`. This intelligently detects whether the issue is Global (like Project Settings) or Scene Local (like a Reflection Probe).

For Scene fixes, it:

1. Locates the GameObject by name.
2. Hard-sets the correct optimized Unity property (e.g. `ReflectionProbe.refreshMode = OnAwake`).
3. Calls `EditorSceneManager.MarkSceneDirty()` to ensure Unity prompts the user to save the corrected scene.

Note: The tool will never Auto-Fix architectural (C# layout) issues without manual user intervention, to prevent destructive workflow overwrites. It safely targets hardware options.

---

7. Analytics Parsing & AI Suggestions

Upon pressing "Stop Recording", the `GenerateFinalSuggestions()` method calculates mathematical averages over the session to determine the primary runtime bottleneck.

Analytical Algorithms

The engine uses standard deviation variance tests against your baseline runtime data. It does not blindly flag "Low FPS". It actively correlates parallel spikes:

- If `averageScriptsMs` > 40% of Main Thread -> Flags excessive logic paths (LINQ allocations, `FindObject` polling, deep nested Loops).
- If `averageBatches` > `batchesWarningLimit` -> Flags draw call saturation (Suggests Static Batching / GPU Instancing).
- If `GC Allocation Spike` correlates identically with `FPS Drop` -> Flags an "Instantiation Memory Leak".
- If `endBatteryLevel` - `startBatteryLevel` > 3% in < 5 mins -> Flags dangerous thermal battery drain code.

These warnings are injected into the top of the resulting `ProfilerReport` ScriptableObject as human-readable strings.

---

8. Enterprise Features (CI/CD Guardian, Wireless, & Comparator)

Wireless Mobile Telemetry (Live TCP Streaming)

If you build your project to an iOS or Android device while it is plugged into your PC via a USB cable, the battery draws power and rapidly overheats the CPU. This causes "thermal throttling," artificially ruining your profiling data. To prevent this, AutoProfiler Pro features a built-in `TcpListener` WebSocket implementation.

Step 1: Network Preparation (Crucial)

Your PC (running the Unity Editor) and your mobile device (Android/iPhone) **MUST be connected to the exact same local Wi-Fi router**. *(For example, if your PC is plugged into the router via Ethernet, and your phone is on that same router's Wi-Fi, that works perfectly).*

Step 2: Configure the Unity Editor

1. In Unity, open the **Auto Profiler Pro** window and go to the 📱 **Wireless** tab.
2. Click 📡 **Start Listening Server**.
3. Note the IP Address displayed on the screen (e.g. `192.168.1.100 : 8080`).

Step 3: Configure Your Scene for the Build

1. In your Unity Scene, select the GameObject that runs your `PerformanceTracker` component.
2. Under the **Wireless Profiling** header in the Inspector:
   - Check `Enable Wireless Profiler` = `True`.
   - Set `Editor IP` = The exact IP from step 2. *(Note: Do NOT use `127.0.0.1` for mobile builds—that is exclusively for Play Mode testing on the same machine).*
   - Set `Editor Port` = `8080`.

Step 4: Build & Play!

1. Go to **File -> Build Settings**.
2. Select **Android** or **iOS**.
3. Click **Build And Run** to install the app on your phone.
4. **Unplug the USB cable** once it finishes to prevent thermal throttling.
5. Watch your Unity Editor! As soon as the app awakes on the phone, the Editor will automatically light up with `Status: 🟢 Connected & Streaming...` and draw live graphs.

⚠ Troubleshooting Windows Firewall

If the app opens on your phone but the Editor stays stuck on 🟡 `Awaiting TCP Connection...`, it means your Windows Defender Firewall is blocking incoming connections to port 8080.

- **The Fix:** When Windows pops up asking if "Unity Editor" is allowed to communicate on Public/Private networks, check both boxes and click **Allow**. Alternatively, manually add an Inbound Rule for Port 8080 in Windows Firewall settings.

A/B Delta Comparator

In the Editor Window, navigate to the ⚖️ **Compare** tab. By dragging in two historical `ProfilerReport` ScriptableObjects (e.g., *Report A* from before optimization, and *Report B* after clicking Auto-Fix), the AI calculates the precise mathematical delta percentages between the sessions. This instantly generates documentation that proves "*You decreased GPU Batches by 14% and freed 256MB of VRAM.*"

Pre-Build Guardian (`IPreprocessBuildWithReport`)

Enabled via the top menu (`Window -> Analysis -> Auto Profiler Guardian`), this script securely hooks into Unity's root compiler via the native interface:

```
class ProfilerBuildGuardian : IPreprocessBuildWithReport {
    public int callbackOrder => 0; // Ensures this runs BEFORE compilation begins
}
```

When a developer attempts to build the game via `Ctrl+B`, the Tool silently wakes up and executes an instantaneous `RunAdvancedEditorAnalysis()`. If any "High Severity" bottlenecks (like 4K bloat or missing LODs) are detected, it throws a `BuildFailedException`. **The Build Stops Completely** and prints an explicit error list to the console. This guarantees that junior developers or unoptimized logic paths never reach a compiled release branch that goes to your QA team or the App Store.

---

9. Advanced Editor Tooling (Heatmaps, Export, Treemaps, & Ghost Scanner)

To completely remove the need for external software, Auto Profiler Pro builds these crucial tools directly into the unified dashboard:

Interactive Build-Size Explorer (The App-Shrinker)

Located in the 📦 **Build Size** tab. Click **"🔍 Scan Project Asset Sizes"** and the tool will iterate through every single file in your `/Assets/` folder, categorize them by type (Textures, Audio, Meshes, Shaders, Other), and calculate the real disk byte-size of each category.

The results are displayed as a **proportional 2D Treemap** — the largest category takes the left half, and the remaining four fill a 2×2 grid on the right. Categories are auto-sorted by size so the worst offenders are always the most visually prominent.

The **"✨ Auto-Crunch Compress All Textures > 2048px"** button actually locates every `TextureImporter` in your project, and if its `maxTextureSize` exceeds 2048 and `crunchedCompression` is disabled, it enables Crunch Compression at quality 50 and reimports the asset. This genuinely reduces your APK/build size.

Shader Complexity & Overdraw Heatmaps

Located in the 💧 **Heatmaps** tab. This tool programmatically switches your Scene View's `DrawCameraMode` via Unity's `SceneView.GetBuiltinCameraMode()` API:

- 💧 **Overdraw Visualization** → `DrawCameraMode.Overdraw` — Highlights transparent pixel stacking that kills GPU fill rate.

- 🦴 **Wireframe Mode** → `DrawCameraMode.Wireframe` — Shows raw mesh topology to identify polygon-heavy objects.
- 🦴 **Shaded Wireframe** → `DrawCameraMode.TexturedWire` — Combines textures with wireframe overlay.
- 🌳 **Restore Original View** → `DrawCameraMode.Textured` — Returns Scene View to standard rendering.

The Scene View updates in real-time. A status indicator tells you whether a diagnostic mode is active.

Ghost Memory & Object Leak Scanner

Located in the 📋 **Leaks** tab. Click **"🔍 Scan for Ghost Objects in Memory"** to run a real-time analysis that:

1. Uses `Resources.FindObjectsOfTypeAll<Texture2D>()` and `AudioClip` to find every asset currently loaded in RAM.
2. Calls `AssetDatabase.GetDependencies()` on the active scene to build a dependency graph.
3. Cross-references loaded assets against scene dependencies — any asset loaded in memory but **not referenced** by the active scene is flagged as a "Ghost Object".
4. For each ghost, you can click **"Ping Object"** to locate it, or **"🧹 Destroy & Free RAM"** to immediately call `Resources.UnloadAsset()` and reclaim the memory.

Deep Project Scanner

Located in the 🛠️ **Deep Scan** tab. Unlike the Offline Scanner (which only checks your active Scene hierarchy), this scans your entire `/Assets/` folder by loading assets via `AssetDatabase.LoadAssetAtPath`. It flags:

- **Massive Audio:** Any `AudioClip` file larger than 5 MB on disk.
- **4K Textures:** Any `Texture2D` with dimensions ≥ 4096px that is silently eating VRAM.

Results are sorted by file size (heaviest first) and each item has a **"Ping in Project"** button that calls `EditorGUIUtility.PingObject()` to jump directly to the asset in your Project window.

Data Exporting (JSON / CSV / Flame Graph)

Located in the 💾 **Export** tab. Three real export pipelines:

- **CSV Export:** Flattens every frame into a spreadsheet row with columns for Time, FPS, CPU, Scripts, Render, Physics, RAM, VRAM, GC, Draw Calls, Batches, Triangles, and Spike status. Opens the file in Finder/Explorer automatically.
- **JSON Export:** Serializes the entire `ProfilerReport` ScriptableObject via `JsonUtility.ToJson()` for CI/CD pipeline integration.
- 🔥 **Flame Graph (.html):** Generates a standalone, self-contained HTML file with inline CSS. The file renders two interactive bar charts (FPS Over Time + CPU Over Time) with color-coded bars (green/yellow/red) and hover tooltips showing per-frame metrics. Open it in any browser and email it directly to stakeholders — no external software required.

---

10. API Reference & Extending the Tool

You can (and should) extend the offline analyzer with rules specific to your studio's game design.

Adding a Custom Rule

1. Open `AutoPerformanceProfiler/Editor/ProfilerAnalyzerExtensions.cs`.
2. Locate the `RunAdvancedEditorAnalysis()` method.
3. Add a custom hierarchy scan block:

```
// Example: Flag all Spheres
var spheres = Object.FindObjectsByType<SphereCollider>(FindObjectsSortMode.None);
foreach(var sphere in spheres)
{
    if(sphere.radius > 50f)
    {
        offenders.Add(new Runtime.ObjectOffender
        {
            gameObjectName = sphere.gameObject.name,
            componentName = "SphereCollider",
            severity = "Medium",
            issueDescription = "Massive hitboxes cause intense broadphase physics
overlap checks.",
            recommendedFix = "Scale the underlying mesh, or break into smaller
compound colliders."
        });
    }
}
```

Adding standard Auto-Fix support for your new rule:

1. Scroll down to `AutoFixSpecificOffender()`.
2. Add an intercept for your logic:

```
if (o.componentName == "SphereCollider")
{
    var sc = target.GetComponent<SphereCollider>();
    if (sc != null)
    {
        sc.radius = 5f; // Cap the size
        EditorSceneManager.MarkSceneDirty(target.scene);
    }
}
```

---

11. Performance Overhead Constraints

- **Runtime Overhead:** < 0.2ms per frame (Predominantly from evaluating threshold booleans).
- **Build Size Impact:** ~50KB. The `UnityEditor` namespace is securely stripped from runtime builds.
- **Screenshot Impact:** Reading pixels from the framebuffer via `ScreenCapture` may cause a microscopic micro-stutter (~10ms) on mobile purely on the *exact frame* the lag spike triggers. Do not leave `takeScreenshotsOnSpike` enabled on highly-optimized, final production Release Builds.

12. Troubleshooting & F.A.Q.

Q: I clicked "Run Offline Scene Scan" but nothing happened? A: Open the Unity Console (`Window -> General -> Console`). Ensure there are no outstanding compiler errors in your own scripts preventing the engine from executing Editor code.

Q: The AI text isn't generating after I click Stop. A: The AI parses the data in the `PerformanceTracker` specifically by analyzing array lengths in the `report.history`. Ensure your session was longer than 5 seconds so enough data points were saved.

Q: Why did my Build fail randomly? A: Check if the **CI/CD Build Guardian** is active (`Window -> Analysis -> Auto Profiler Guardian`). If it is, the Console will list exactly which GameObject is violating architecture conventions. Fix them or disable the tool to proceed with the build.

Q: The Screenshots are coming out black! A: Unity's `ScreenCapture` API requires the Game View to be visible. If you click Record and then minimize the Editor to browse the web, the graphics thread stops rasterizing the UI buffer, resulting in black pixels.

End of Documentation. For support, please reach out to the developer.